Tenderfish - Five Core Specifications Before Build

Developer document covering the five remaining core areas: object model, state machine, approval matrix, readiness rules, and role permissions. This document combines source-informed patterns from public materials with explicit product recommendations where final Tenderfish decisions are still required.

Specification area	Intent
Object model	Define the Phase 1 entities, relationships, and ownership rules.
State machine	Define how a project moves from raw intake to formal tender release.
Approval matrix	Define where human release is mandatory and who is allowed to approve.
Readiness rules	Define measurable gates for structure, cost, detail planning, and tender release.
Role permissions	Define object-level access for admins, architects, team members, vendors, clients, and external sources.

Source-informed patterns used here

- ORCA AVA public handbook material supports a formal project, package, document, cost, and tendering logic.

- CarbonSpace supports a clean project-version-inventory-materials-report pattern for structured state handling.

- MAIER CONTROL and baufaktura support participant, field input, browser administration, documents, reports, and operational object patterns.

- Where the public sources do not dictate one single answer, this document marks the recommendation as a Tenderfish best-practice proposal.

1. Binding object model

This section locks the Phase 1 first-class objects. The goal is to keep the MVP narrow, auditable, and tender-focused. The recommended model takes the project-version pattern from CarbonSpace and combines it with ORCA-like tender and document logic plus MAIER-like participant and operational record logic.

1.1 Phase 1 first-class objects

Object	Purpose	Create mode
Project	Top-level container for all intake, structure, approvals, and tender outputs	Manual
Project Version	Frozen or active version of project state	Manual plus system-assisted
Structured Project Description	Approved narrative and factual definition of current project scope	System-generated, then approved
Approved Resource	Any document, drawing, file, or source accepted as part of project truth	System-detected, then approved

Object	Purpose	Create mode
Document	General uploaded or captured file record with provenance	Automatic or manual
Participant	Person or company linked to the project	Automatic inference plus manual confirmation
Approval	Formal review or release action with audit trace	System-created from rules
Cost Group	Structured commercial grouping	System-generated then editable
Cost Item	Atomic commercial item inside a cost group or package	System-generated then editable
Package	Tender package or procurement grouping	System-generated then editable
Tender Release State	Explicit end-state object representing formal release readiness	System-created from rules and approval

1.2 Object relationships

- A Project contains many Project Versions.

- A Project Version contains one Structured Project Description, many Approved Resources, many Cost Groups, many Cost Items, many Packages, and many Approvals.

- Documents may exist before approval, but only Approved Resources count toward formal project truth.

- Participants belong to the Project and may be linked to Packages, Documents, Approvals, or communication history.

- Tender Release State belongs to a single Project Version and cannot exist without approval prerequisites.

1.3 Best-practice proposal

For Phase 1, do not make every possible artifact a first-class object. Keep comments, inbox items, confidence flags, and source traces as secondary records attached to the first-class objects above. This preserves simplicity while retaining auditability.

2. State machine

The public tools suggest strong structured progression, but Tenderfish needs its own explicit state chain because the MVP ends at formal tender release. The recommended state machine below is intentionally strict, so architects retain control while the system performs most of the heavy lifting.

State	Meaning	Typical trigger
input_received	Raw input exists in intake engine	Upload, email, message, bot note
parsed	System has extracted candidate structure	Automatic parser/classifier
needs_review	Human review required before project truth	Low confidence or rule requirement
confirmed	Candidate accepted by responsible user	Manual confirmation

State	Meaning	Typical trigger
structure_approved	Structured project description and approved resources are formally accepted	Architect or lead approval
cost_ready	Cost estimate can be generated from approved state	Rule-based gate passed
detail_ready	Detailed planning and package structure are sufficient for tender prep	Rule-based gate passed
tender_ready	All release prerequisites are complete	Readiness rules passed
released_for_tender	Tender package is formally released	Explicit human release action

2.1 Transition rules

- No object may move directly from input_received to confirmed without parse output or human review.

- Only approved resources may count toward structure_approved.

- cost_ready and detail_ready may be system-calculated, but tender_ready must remain reviewable.

- released_for_tender must always require an explicit human action.

2.2 Best-practice proposal

Use soft automation early and hard gates late. In other words, parsing and candidate creation can be highly automated, but structure approval and tender release should remain human-controlled gates with audit history.

3. Approval matrix

Approvals are the control layer that prevents automation from turning into loss of governance. Public references support signatures, acceptance records, browser-side administration, and structured project governance, but the exact matrix is a Tenderfish design decision. The following is the recommended Phase 1 matrix.

Object / decision	Who can approve	Is approval mandatory	Unlocks
Structured Project Description	Architect or Project Lead	Yes	structure_approved
Approved Resource	Architect or Project Lead	Yes for formal relevance	counts toward readiness
Participant role confirmation	Admin or Architect or Project Lead	Yes where permissions matter	role-based access
Cost Estimate State	Architect or Project Lead	Yes	cost_ready sign-off
Package Definition	Architect or Project Lead	Yes	detail_ready
Tender Release State	Architect or Project Lead	Yes	released_for_tender

3.1 Approval behavior

- Every approval must store actor, timestamp, object reference, previous status, new status, and optional comment.

- Approvals should be visible as clear buttons or action bars in the UI, not hidden in menus.

- Decline, rework, and request-clarification are as important as approve.

3.2 Best-practice proposal

Use one consistent approval component across the platform. Even if the object changes, the user should recognize the pattern: review context, reason, confidence or completeness summary, approve or send back.

4. Readiness rules

Readiness is not just a visual meter. It must be a rule-based project gate. The sources support structured progression, but the exact thresholds must be fixed by Tenderfish. The safest approach is a phase-based rule set with transparent missing items.

4.1 Proposed readiness levels

Readiness level	Minimum rule set
Structured Project Description Readiness	Project description exists, approved resources selected, open points captured, responsible participants identified
Cost Estimate Readiness	Structure approved, cost groups created, cost items present, minimum required approved resources present
Detailed Planning Readiness	Package structure defined, supporting descriptions present, relevant documents linked, unresolved critical gaps reduced to allowed threshold
Tender Readiness	All mandatory approvals complete, required approved resources complete, cost estimate signed off, package definition complete, tender release object available

4.2 UI behavior

- Each readiness score should list exactly what is missing.

- The user must be able to click from the readiness module directly to the missing objects.

- Readiness should distinguish between hard blockers and soft improvement suggestions.

4.3 Best-practice proposal

Treat readiness as a checklist-backed score, not a black-box percentage. Users should trust the number because they can inspect its underlying rules.

5. Role and permission matrix

The public sources strongly support differentiated administration and field access, but Tenderfish needs an object-level matrix to avoid ambiguity. The recommended approach combines role-by-project with object-by-action rights.

Role	Core position in Phase 1	Typical rights
Admin / Owner	System and project governance	All project configuration, participant management, exports, approvals where delegated
Architect / Project Lead	Primary accountable role	Review, edit, approve, release, structure, cost, package, export
Internal Team Member	Supports preparation	Upload, comment, edit selected objects, no final tender release

Role	Core position in Phase 1	Typical rights
Vendor / Supplier	May provide documents or later quotes	Limited upload and view only where invited, no internal cost control rights
Client	May review approved outputs	View approved outputs, comment where allowed, no editing of core structure
External Source	No real system role yet	Contribute information only, no project navigation

5.1 Object-level action rules

- Only Admin and Architect / Project Lead may approve Structured Project Description and Tender Release State.

- Internal Team Members may help curate documents, draft structure, and prepare package logic, but should not perform final release.

- Vendor / Supplier should remain mostly outside the MVP core until invited into tender participation or later phases.

- Client access should default to approved and intentionally shared outputs only.

5.2 Best-practice proposal

Implement permissions as project + role + object + action, not as one global user type. This preserves flexibility and prevents later rebuilds when collaboration becomes more complex.

6. Where the recommendations are strongest and where they are still product choices

The following are strongly supported by the public materials we reviewed:

- Versioned project structure

- Formal distinction between input and approved state

- Role separation between administration and lighter participant access

- Operational importance of documents, reports, and provenance

- Human-controlled approvals before formal release

The following remain explicit Tenderfish design choices:

- Exact mandatory fields of Structured Project Description

- Exact numerical readiness scoring logic

- Final list of Phase 1 exports

- Whether some review actions belong to Admin only or Architect and Project Lead together

The safest implementation strategy is to keep these as locked configuration choices rather than leaving them implicit in code.

7. Recommended next step

This document is strong enough to guide implementation. The next practical step is to turn each of the five sections into one implementation matrix or schema sheet.

- Object model -> database objects and relationships

- State machine -> statuses, transitions, triggers

- Approval matrix -> approver, action, consequence

- Readiness rules -> checklist-backed gate logic

- Role permissions -> API and UI access matrix

Short internal summary

Yes, the public materials are sufficient to create a strong build document for these five areas. Where the sources are silent, this specification uses best-practice proposals designed to maximize architect control while offloading as much structuring and review work as possible to the system.